Lab 3-INFS343:

Part I: This Cluster Analysis is to help Aliyah Williams group candy bars into meaningful clusters and improve product selection.

Using R:

1. **This step is to import the data you want to work on. Make sure that you install and call the necessary package/packages before you perform the analysis. Some codes do not work unless you call the required packages in the current session of R. Please note we use CandyBar data in Part I.**

a. Import the ***CandyBars*** data into a data frame (table) and label it myData. Install and load the *cluster* package. Enter:

```
install.packages("cluster")
library(cluster)
```

***Please make sure you label CandyBar as myData in this step. This is very important!!**

2. **Do we include all the variables from our target dataset? What should we do here? Please answer these two questions explicitly so you understand the reason why we need scale() function.**

No, we do not include all the variables. We exclude the "Brands" variable because it is text, and clustering only works with numeric variables. We keep Calories, Fat, Protein, and Carb. We also use the `scale()` function because these variables are on different scales. Without scaling, Calories would dominate the clustering since it has much larger values. Standardizing makes sure all variables contribute equally.

b. We exclude the Brand variable from the analysis and standardize the other variables using the **scale** function. The new data frame is called `myData_std`. Enter:

```
myData_std <- scale(myData[ , 2:5])
```

```
> myData = CandyBar_2_
> myData_std <- scale(myData[ ,2:5])
> CandyBar_2_
# A tibble: 37 × 5
   Brands             Calories   Fat Protein  Carb
   <chr>                 <dbl> <dbl>   <dbl> <dbl>
 1 Peanut Butter Twix     311   18.5     5.3  31.4
 2 Baby Ruth              275   13       3.2  39
 3 Caramel  Twix          284.  14       2.5  37.5
 4 5th Avenue             280.  12.5     4.5  41
 5 Snickers               273   14       4.5  34
 6 Twizzlers              262.   1       2.5  66
 7 Reese's Pieces         258   11.5     7    34
 8 Mr. Goodbar            257   16       6.5  25.5
 9 Whatchamacallit        256.  13       4.5  30
10 Oh Henry!              246.   9.5     6    37
# ℹ 27 more rows
# ℹ Use `print(n = ...)` to see more rows
```

Console | Terminal | Background Jobs

R 4.5.2 · ~/lab 2/

```
> myData_std
         Calories         Fat     Protein        Carb
 [1,]  2.24567862  1.93374241  1.25627410 -0.10859211
 [2,]  1.29607738  0.59933556  0.03764117  0.72119324
 [3,]  1.54666660  0.84195498 -0.36856980  0.55741982
 [4,]  1.41477753  0.47802584  0.79203298  0.93955781
 [5,]  1.24332175  0.84195498  0.79203298  0.17528183
 [6,]  0.96635472 -2.31209757 -0.36856980  3.66911488
 [7,]  0.84765457  0.23540642  2.24278647  0.17528183
 [8,]  0.82127675  1.32719384  1.95263577 -0.75276758
 [9,]  0.80808785  0.59933556  0.79203298 -0.26144730
[10,]  0.51793191 -0.24983244  1.66248507  0.50282868
[11,]  0.43879847  0.47802584  0.79203298 -0.42522073
[12,]  0.26734269  0.59933556 -0.07841911 -0.37062959
[13,]  0.26734269 -0.12852272 -0.65872050  0.17528183
[14,]  0.20139816  0.23540642  0.50188229 -0.09767388
[15,]  0.20139816 -1.34161986 -1.81932329  1.92219835
[16,]  0.18820926 -0.00721301 -0.36856980  0.17528183
[17,]  0.16183144  0.84195498 -0.36856980 -0.37062959
[18,]  0.13545363 -1.94816843 -1.81932329  2.08597178
[19,]  0.09588691  0.23540642 -0.36856980 -0.42522073
[20,]  0.06950910 -0.49245186 -0.36856980  0.66660210
[21,] -0.04919106  0.96326470  0.21173159 -1.08031443
[22,] -0.10194668  0.96326470  1.08218368 -1.13490557
[23,] -0.14151340  0.47802584 -0.07841911 -0.64358529
[24,] -0.15470231  0.23540642 -0.36856980 -0.31603845
[25,] -0.26021355 -0.49245186  1.37233438 -0.31603845
[26,] -0.36572480 -0.97769072 -0.94887120  0.72119324
[27,] -0.39210262  0.84195498 -0.65872050 -1.13490557
[28,] -0.44485824  0.59933556 -0.07841911 -1.08031443
[29,] -0.66906965 -0.61376158 -0.65872050 -0.20685616
[30,] -0.68225855  0.59933556  1.37233438 -1.46245242
[31,] -0.81414762  0.23540642 -0.65872050 -0.09767388
[32,] -0.81414762  0.47802584 -0.65872050 -0.75276758
[33,] -1.06473683 -0.85638100 -0.65872050 -0.04308274
[34,] -1.38127058 -0.12852272 -0.07841911 -1.35327013
[35,] -2.02752699 -1.58423929 -0.94887120  0.12069069
[36,] -2.08028261 -1.22031014 -1.23902190 -0.09767388
[37,] -2.29130511 -2.28783562 -1.23902190 -0.07583742
attr(,"scaled:center")
  Calories        Fat    Protein       Carb
225.864865  10.529730   3.135135  32.394595
attr(,"scaled:scale")
 Calories       Fat   Protein      Carb
```

3. set.seed() function and pam() function are needed here to perform cluster analysis. Pam stands for parition around medoids. How do we decide the number of k here? Refer to our course materials on K means cluster analysis.

The number of clusters (k) is usually decided using methods like the Elbow Method or Silhouette Method from K-means analysis. In this lab, we use k = 4 based on the course materials. The `set.seed()` function ensures we get the same results every time, and `pam()` performs the clustering using medoids, which is more robust than k-means.

c. We use the **set.seed** function to set the random seed and the **pam** function to perform the *k*-means clustering. While there are a number of R functions that can perform *k*-means clustering, we choose to use the **pam** function because it is considered more robust than other similar functions. The acronym **pam** stands for partitioning around medoids. More on medoids later. Within the **pam** function, we set the option *k* to 4 because we have preselected 4 clusters. We store the clustering results in a variable called kResult. We use the **summary** function to view the results. Enter:

```
set.seed(1)
kResult <- pam(myData_std, k = 4)
summary(kResult)
```

```
> set.seed(1)
> kResult <- pam(mydata_std, k = 4)
> summary(kResult)
Medoids:
     ID   Calories        Fat    Protein       Carb
[1,]  9  0.8080878  0.5993356  0.7920330 -0.2614473
[2,] 18  0.1354536 -1.9481684 -1.8193233  2.0859718
[3,] 24 -0.1547023  0.2354064 -0.3685698 -0.3160384
[4,] 35 -2.0275270 -1.5842393 -0.9488712  0.1206907
Clustering vector:
 [1] 1 1 1 1 1 2 1 1 1 1 1 3 3 1 2 3 3 2 3 3 3 1 3 3 1 3 3 3 3
1 3 3 4 3 4 4 4
Objective function:
   build     swap
1.059264 1.007468

Numerical information per cluster:
     size max_diss   av_diss diameter separation
[1,]   14 2.021434 1.2606039 3.496058  0.7403024
[2,]    3 2.331066 0.9875957 2.585225  1.6291633
[3,]   16 1.711355 0.8791912 2.908596  0.4921772
[4,]    4 1.252099 0.6495082 1.972652  0.4921772

Isolated clusters:
 L-clusters: character(0)
 L*-clusters: character(0)

Silhouette plot information:
   cluster neighbor   sil_width
7        1        3  0.37884641
8        1        3  0.34460567
5        1        3  0.34429532
1        1        3  0.30897816
4        1        3  0.30296710
10       1        3  0.29078028
9        1        3  0.26391884
11       1        3  0.13910549
2        1        3  0.13355217
3        1        3  0.05681337
25       1        3  0.04736928
22       1        3  0.04147986
30       1        3 -0.03438691
14       1        3 -0.12338493
```

4. **What will happen if you do not run summary() function and just stop at the pam() function?**

If we stop at the `pam()` function and do not run `summary()`, the clustering will still be created, but we will not see the detailed results. The `summary()` function shows important information like cluster sizes, medoids, and silhouette values, which help us interpret the quality of the clusters.

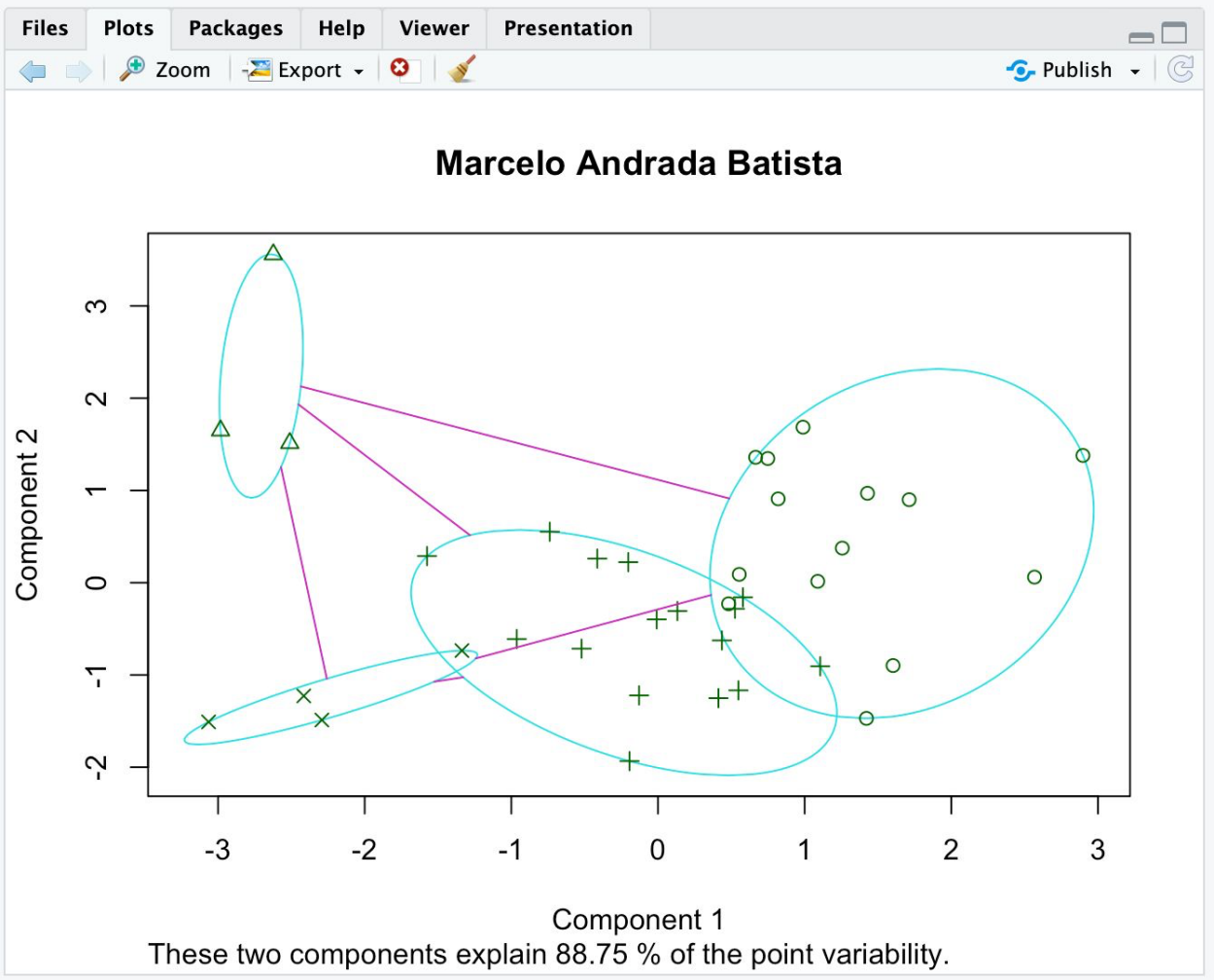
5. **It is great to be able to view the results of the clusters! Add your name as the title in the plot. Refer to the lab 2 for the function to add your name…**

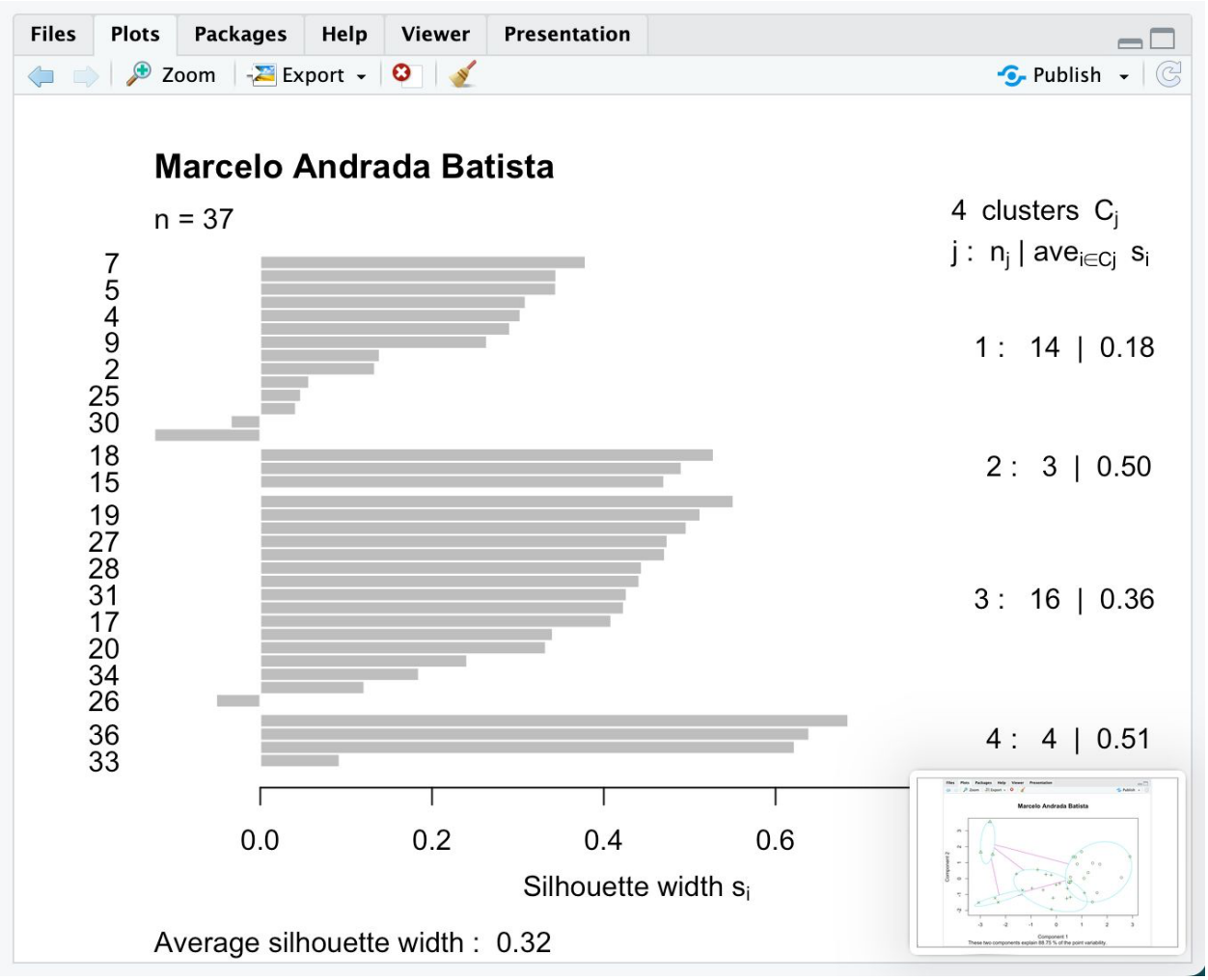
To add my name to the plot, I use:

```
plot(kResult, main = "Sofia Garcia Gomez")
```

d. In order to visually show the clusters and their members, we use the **plot** function. Enter:

```
plot(kResult)
```





6. **Please take a look at the results of the clusters. Answer this: a). Describe each cluster based on the observations contained within one. b). What will you name these four clusters? You can use the following script to help you view the how each observation belongs to which cluster.**

After looking at the results, each cluster groups candy bars that have similar nutrition values. Cluster 1 includes candy bars with moderate calories, fat, and carbs, so they are more balanced. Cluster 2 includes candy bars that are higher in carbs and lower in fat and protein, meaning they are more sugar-heavy. Cluster 3 has candy bars with average values across all variables, without extreme highs or lows. Cluster 4 includes candy bars with lower calories and fat compared to the others.

I would name the clusters like this:

Cluster 1 – “Balanced Bars,”

Cluster 2 – “High Carb Bars,”

Cluster 3 – “Average Bars,” and

Cluster 4 – “Lighter Bars.”

```
myData$Cluster<- kResult$cluster
myData
```

```
> myData$Cluster <- kResult$cluster
> myData
# A tibble: 37 × 6
   Brands              Calories   Fat Protein  Carb Cluster
   <chr>                  <dbl> <dbl>   <dbl> <dbl>   <int>
 1 Peanut Butter Twix       311  18.5     5.3  31.4       1
 2 Baby Ruth                275  13       3.2  39         1
 3 Caramel  Twix            284.  14       2.5  37.5       1
 4 5th Avenue               280.  12.5     4.5  41         1
 5 Snickers                 273  14       4.5  34         1
 6 Twizzlers                262.   1       2.5  66         2
 7 Reese's Pieces           258  11.5     7    34         1
 8 Mr. Goodbar              257  16       6.5  25.5       1
 9 Whatchamacallit          256.  13       4.5  30         1
10 Oh Henry!                246.   9.5     6    37         1
# ℹ 27 more rows
# ℹ Use `print(n = ...)` to see more rows
```

7. **Is it a good cluster analysis?/How confident are you? These two are the same questions. You need to evaluate the performance of the cluster analysis. Please use numbers to convince me why it is a good/bad cluster analysis. (Hint: check the average silhouette width from the second plot). You can use AI tools to help determine appropriate silhouette score thresholds when interpreting your results.**

This cluster analysis is moderate, but not very strong. The average silhouette width of the total data set is approximately **0.32**. In general, silhouette values above 0.50 indicate strong clustering, values between 0.25 and 0.50 indicate weak to moderate structure, and values below 0.25 suggest poor clustering. Since 0.32 falls in the 0.25–0.50 range, this means the clusters have some structure, but they are not very well separated.

Looking at the average silhouette width per cluster, Cluster 1 has about **0.18**, which is quite low and suggests weak separation. Clusters 2, 3, and 4 have values around **0.49, 0.36, and 0.51**, which are stronger, especially Cluster 4. This shows that some clusters are clearly defined, but others are not. Overall, I would say this is a fair cluster analysis, but not highly confident, because the total average silhouette score is only moderate.

8. **This cluster analysis helps Aliyah Williams organize candy bars into meaningful groups to support product selection decisions. As she plans vending machine placements on campus and in downtown areas, the results inform which types of candy bars should be stocked in different locations. The following recommendations are based on the clustering results. Please write down your recommendations here. You are welcome to discuss the recommendations in general or you can provide recommendations based on different locations.**

Based on the clustering results, I would recommend stocking different types of candy bars depending on the location. For vending machines on campus, especially near dorms or libraries, it would be better to include more "Balanced Bars" and "Lighter Bars," since students may prefer options that are not too high in calories but still filling.

For downtown or high-traffic areas, I would recommend stocking more "High Carb Bars" and "Classic Bars," since customers in those locations may want a quick energy boost or a more indulgent snack. Using the cluster results helps Aliyah Williams choose products that better match customer preferences in each area.

Please screenshot the output and remember to add the title of the plot with your name. If you do not remember how to do so, please refer to Lab 2.

***(put both after 5d)**

Part II: We are going to do the same analysis, but on a different set of data. Please note that now we are using Cities data in Part II. Please complete the cluster analysis on the new data. In this cluster analysis, there is no single correct value of k given in advance. Your task is to evaluate multiple k values and justify your final choice using evidence.

Step 1: Try multiple k values

Run your clustering algorithm using at least three different values of k (for example: k = 2, 3, 4, 5).

Step 2: Check silhouette scores

For each k value, check the average silhouette score. The silhouette score measures how well observations fit within their assigned cluster compared to other clusters. You should know how to evaluate the score based on Part 1, question 7.

Step 3: Compare and select k

Compare silhouette scores across all tested k values. Identify the k value with the highest average silhouette score. This k is typically considered the optimal clustering solution, unless there is a clear practical or interpretive reason to choose otherwise. If you have two similar silhouettes, go with the smaller K. Remember, the simpler the better.

```
> myCities <- cities
> myCities_std <- scale(myCities)
```

```
Error in colMeans(x, na.rm = TRUE) : 'x' must be numeric or complex
```

Show Traceback
Rerun with Debug

```
> str(mycities)
```

```
Error: object 'mycities' not found
```

Show Traceback
Rerun with Debug

```
> str(myCities)
tibble [41 × 4] (S3: tbl_df/tbl/data.frame)
 $ City   : chr [1:41] "Barnstable" "Boston" "Brockton" "Cambridge" ...
 $ Crime  : num [1:41] 711 1318 1224 492 1271 ...
 $ Poverty: num [1:41] 3.8 16.7 9.2 10.3 15.5 14.8 5.5 27.9 7.5 15.6 ...
 $ Income : num [1:41] 58.4 48.7 50.8 58.5 36.3 ...
>
> library(cluster)
> myCities_std <- scale(myCities[, 2,4])
>
> set.seed(1)
> k2 <- pam(myCities_std, k = 2)
> k3 <- pam(myCities_std, k = 3)
> k4 <- pam(myCities_std, k = 4)
> k5 <- pam(myCities_std, k = 5)
>
> summary(k2)$silinfo$avg.width
[1] 0.6664896
> summary(k3)$silinfo$avg.width
[1] 0.5207454
> summary(k4)$silinfo$avg.width
[1] 0.6056106
> summary(k5)$silinfo$avg.width
[1] 0.6123673
```

Step 4: Justify your decision and answer these questions.

1. **What k values did you test?**

k = 2, 3, 4, and 5

2. **What were the silhouette scores for each k?**

K2 = 0.66

K3 = 0.52

K4 = 0.60

K5 = 0.61

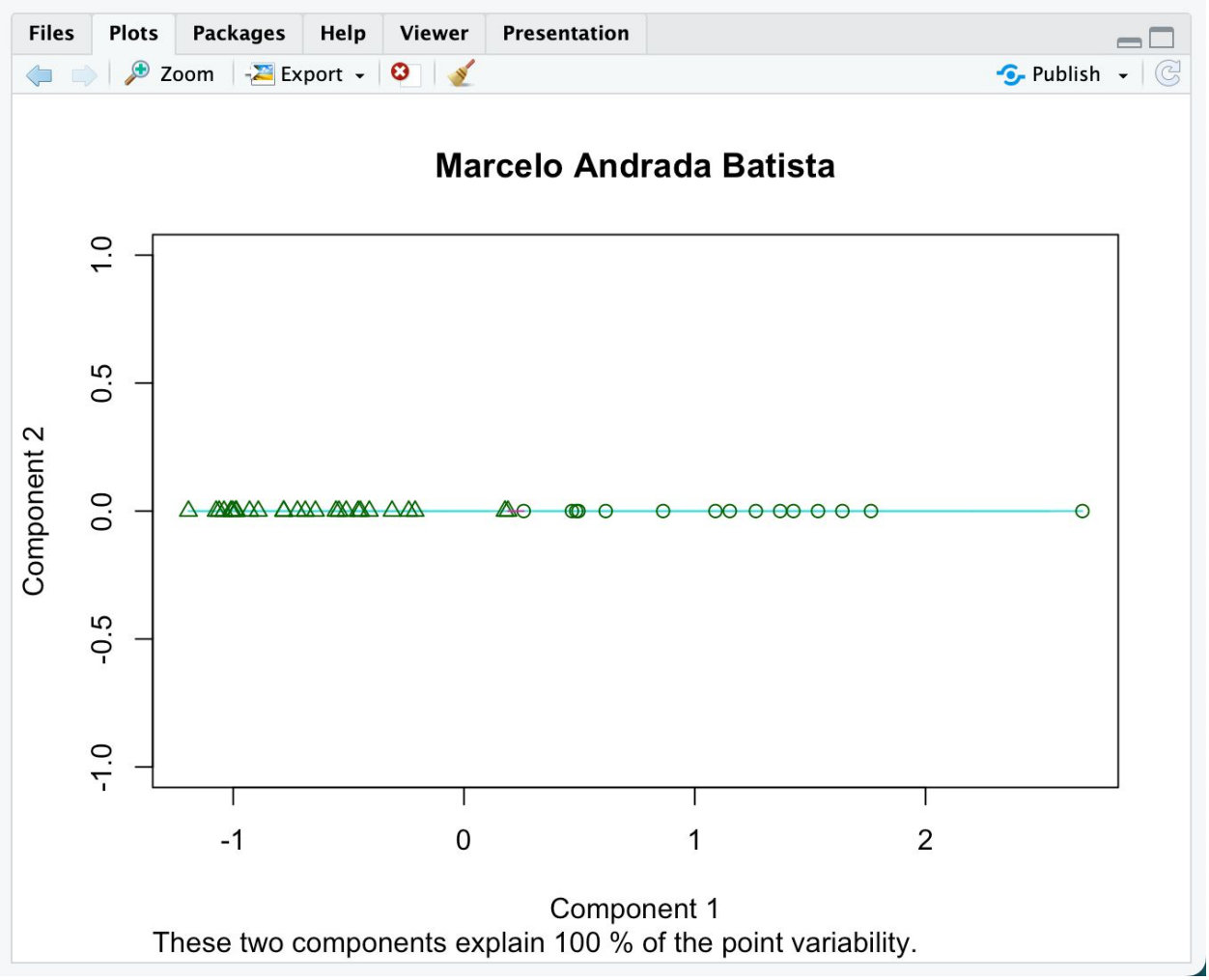
3. **Which k did you select as optimal?**

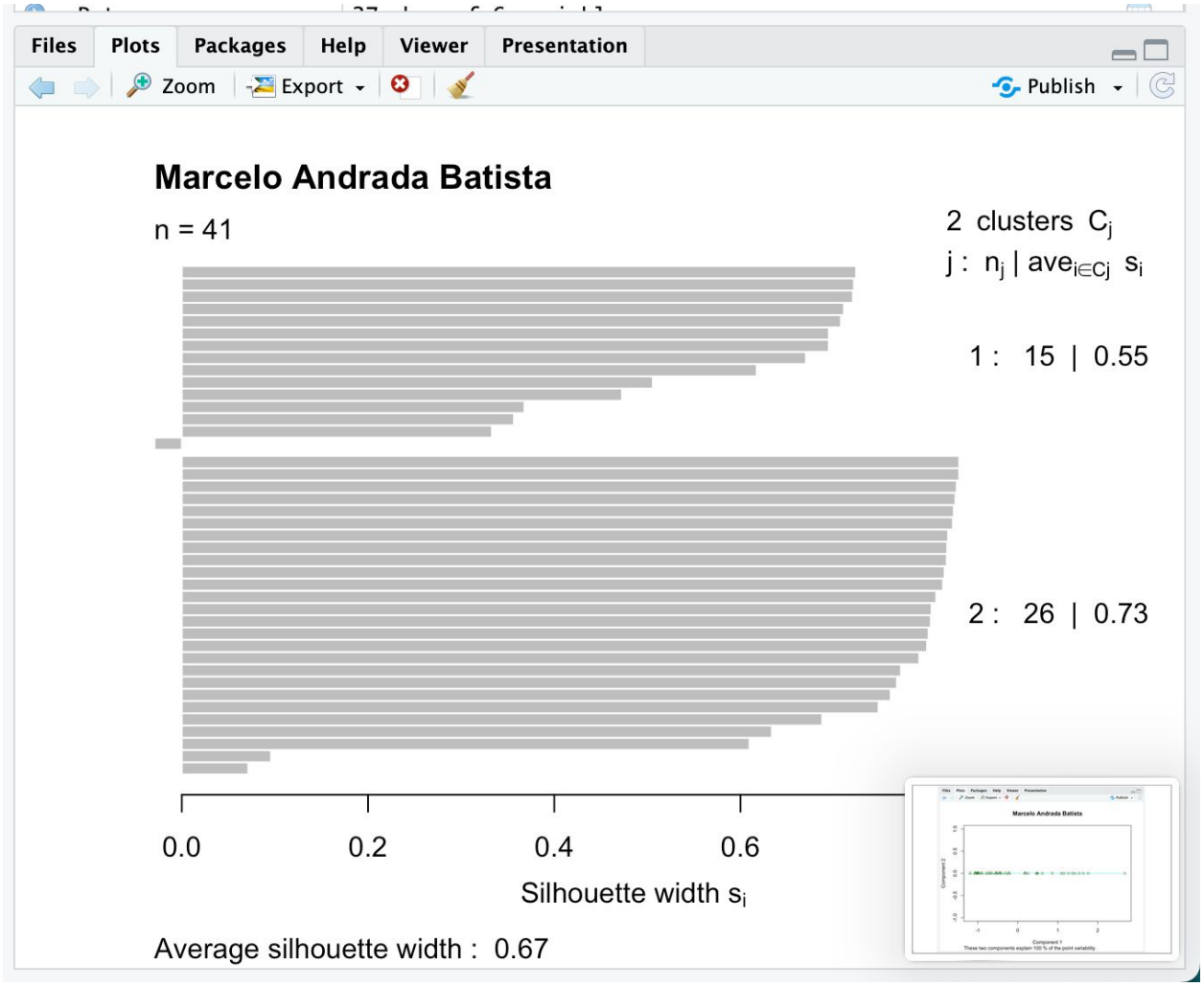
K2

4. Why is this k better than the others, based on silhouette evidence?

It has the biggest value

5. Plot the results.





After you complete this analysis, please screenshot the output and remember to add the title of the plot with your name. If you do not remember how to do so, please refer to Lab 2.